

 	Departamento: CIENCIAS DE LA COMPUTACION
	Carrera: Electrónica Y Automatización Asignatura: Fundamentos De Programación NRC: 28561
	Apellidos y nombres del estudiante: Angamarca Cuchiye Lenin Jhosue

EJERCICIOS DE VECTOR Y MTZ

1. Ejercicios de vectores

Ejercicio 2.1.1. Vector al revés

Enunciado. Desarrolle un programa que lea las N componentes de un vector y las escriba a continuación en orden inverso al introducido. El valor de N se introduce por teclado y se supone que será igual o menor a 10.

Idea clave. Leer el vector en orden normal y luego recorrerlo desde la última posición hasta la primera.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Vector de 10 componentes	vec	var	real
Contador	i	var	entero
Uno	1	cte	entero
Numero de componentes del vector	N	var	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué el segundo recorrido debe decrementar el índice? ¿Qué pasaría si se imprime nuevamente de 1 a N?	

Análisis del problema

El programa debe leer los elementos de un vector y luego mostrarlos en orden inverso. La salida esperada es el vector desde la última posición hasta la primera.

Algoritmo VectorAlReves

Definir vec[10] Como Real
 Definir i, N Como Entero

Escribir "Ingrese tamaño del vector:"
 Leer N

```

Para i <- 0 Hasta N-1 Hacer
    Escribir "Ingrese elemento:"
    Leer vec[i]
FinPara

```

```

Escribir "Vector invertido:"

```

```

Para i <- N-1 Hasta 0 Con Paso -1 Hacer
    Escribir vec[i]
FinPara

```

```

FinAlgoritmo

```

Prueba de Escritorio:

i	vec[i] ingresado	Salida
0	5	
1	8	
2	2	
3	9	
3		9
2		2
1		8
0		5

Codificación en C:

```

#include <stdio.h>

```

```

int main() {

```

```

    float vec[10];
    int i, N;

```

```

    printf("Ingrese tamaño del vector: ");
    scanf("%d", &N);

```

```

    for(i = 0; i < N; i++) {

```

```

        printf("Ingrese elemento: ");
        scanf("%f", &vec[i]);
    }

```

```

    printf("Vector invertido:\n");

```

```

    for(i = N - 1; i >= 0; i--) {
        printf("%.2f\n", vec[i]);
    }

```

```

    return 0;
}

```

Reflexión Crítica:

Se usa un recorrido decreciente porque se necesita mostrar el vector desde el último elemento hasta el primero.

Si el segundo ciclo fuera de 0 hasta N-1, el vector se imprimiría nuevamente en orden normal.

Ejercicio 2.1.2. Norma de un vector al cuadrado

Enunciado. Desarrolle un programa que lea las n componentes reales de un vector vec y calcule la norma al cuadrado: $m = \text{vec}^T \times \text{vec} = \text{suma de } \text{vec}(i)^2$. El tamaño n se introduce por teclado y puede suponerse igual o menor que 10.

Idea clave. Inicializar res en 0 y acumular el cuadrado de cada componente leída.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Vector de 10 componentes	vec	var	real
Contador	i	var	entero
Número de componentes del vector	n	var	entero
Norma al cuadrado	res	var	real
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica que datos se leen por teclado y cuales ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	

Objeto	Nombre	Valor	Tipo
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué crees debe iniciar en cero? ¿Qué error aparecería si se usa un valor basura no inicializado?	

Análisis del problema

El programa debe calcular la suma de los cuadrados de cada elemento del vector.

Algoritmo NormaCuadrado

Definir vec[10] Como Real

Definir i, n Como Entero

Definir res Como Real

```
res <- 0
```

```
Escribir "Ingrese tamaño:"
```

```
Leer n
```

Para i <- 0 Hasta n-1 Hacer

Leer vec[i]

res <- res + vec[i]^2

FinPara

Escribir "Resultado: ", res

FinAlgoritmo

Prueba de Escritorio:

i	vec[i]	Operación	res
0	2	$0 + 2^2$	4
1	3	$4 + 3^2$	13
2	4	$13 + 4^2$	Salida: 29

Codificación en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    float vec[10], res = 0;
```

```
    int i, n;
```

```
    printf("Ingrese tamaño: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++) {
```

```

scanf("%f", &vec[i]);

res = res + (vec[i] * vec[i]);

}

printf("Norma al cuadrado = %.2f", res);

return 0;

}

```

Reflexión crítica

La variable res debe iniciar en cero porque acumula resultados.
Si no se inicializa, puede contener basura de memoria y el resultado sería incorrecto.

Ejercicio 2.1.3. Vector con término general dado

Enunciado. Sea la sucesión $v_k = k^2 + 3$. Desarrolle un programa que lea el número n de componentes que se quieren de la sucesión y las almacene en un vector vec, de forma que $vec(i) = v_i$. Puede asumir que n será menor o igual a 100.

Idea clave. Usar un ciclo para calcular cada término de la sucesión y almacenarlo en la posición correspondiente del vector.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Vector de 100 componentes	vec	var	real
Contador	i	var	entero
Uno	1	cte	entero
Número de componentes del vector	n	var	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuales ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Qué diferencia existe entre el índice usado en el enunciado y el índice que usa C cuando empieza en 0?	

Análisis del problema

El programa debe generar una sucesión usando la fórmula $vk = k^2 + 3$ y almacenarla en un vector.
Algoritmo Sucesion

Definir vec[100] Como Real
Definir i, n Como Entero

Leer n

Para i <- 0 Hasta n-1 Hacer

vec[i] <- (i*i) + 3

FinPara

Para i <- 0 Hasta n-1 Hacer

Escribir vec[i]

FinPara

FinAlgoritmo

Prueba de Escritorio:

i	Operación	vec[i]
0	$0^2 + 3$	3
1	$1^2 + 3$	4
2	$2^2 + 3$	7
3	$3^2 + 3$	12

Codificación en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    float vec[100];
```

```
    int i, n;
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++) {
```

```
        vec[i] = (i * i) + 3;
    }
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("%.2f\n", vec[i]);
    }
```

```
    return 0;
```

```
}
```

Reflexión crítica

En C los índices comienzan en 0 y no en 1.

Por eso el primer valor calculado es cuando $i = 0$.

Ejercicio 2.1.4. Comprobar si dos valores pertenecen a un vector

Enunciado. Realice un algoritmo que lea dos números enteros por teclado y determine si ambos

valores forman parte de un vector de enteros previamente definido de dimensión 15.

Idea clave. Usar dos variables bandera para marcar si num1 y num2 se encuentran en el vector.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Número de componentes del vector	N	cte	entero
Vector de 15 números	vec	var	entero
Variable auxiliar para el primer dato	aux1	var	entero
Variable auxiliar para el segundo dato	aux2	var	entero
Contador	i	var	entero
Primer dato de entrada	num1	var	entero
Segundo dato de entrada	num2	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué se necesitan dos banderas y no una sola? ¿Qué condición final permite afirmar que ambos números están en el vector?	

Análisis del problema

El programa debe leer dos números enteros y verificar si ambos pertenecen a un vector de 15 elementos.

Algoritmo BuscarNumeros

```

Definir vec[15] Como Entero
Definir num1, num2 Como Entero
Definir aux1, aux2 Como Entero
Definir i Como Entero

```

```

aux1 <- 0
aux2 <- 0

```

```

Para i <- 0 Hasta 14 Hacer
    Leer vec[i]
FinPara

```

```

Escribir "Ingrese primer numero:"
Leer num1

```

```

Escribir "Ingrese segundo numero:"
Leer num2

```

```

Para i <- 0 Hasta 14 Hacer

    Si vec[i] = num1 Entonces
        aux1 <- 1
    FinSi

```

```

    Si vec[i] = num2 Entonces

```

```

        aux2 <- 1
    FinSi

FinPara

Si aux1 = 1 Y aux2 = 1 Entonces
    Escribir "Ambos pertenecen"
Sino
    Escribir "Uno o ambos no pertenecen"
FinSi

```

FinAlgoritmo

Prueba de Escritorio:

i	vec[i]	aux1	aux2
0	2	0	0
1	4	1	0
2	6	1	0
3	8	1	1
			Salida: Ambos pertenecen

Codificacion en C:

```

#include <stdio.h>

int main() {

    int vec[15];
    int num1, num2;
    int aux1 = 0, aux2 = 0;
    int i;

    for(i = 0; i < 15; i++) {

        printf("Ingrese elemento: ");
        scanf("%d", &vec[i]);
    }

    printf("Ingrese primer numero: ");
    scanf("%d", &num1);

    printf("Ingrese segundo numero: ");
    scanf("%d", &num2);

    for(i = 0; i < 15; i++) {

        if(vec[i] == num1) {
            aux1 = 1;
        }
    }
}

```



```

        if(vec[i] == num2) {
            aux2 = 1;
        }
    }

    if(aux1 == 1 && aux2 == 1) {
        printf("Ambos pertenecen");
    } else {
        printf("Uno o ambos no pertenecen");
    }

    return 0;
}

```

Reflexión crítica

Se utilizan dos banderas porque cada número debe verificarse por separado.
Con una sola bandera no se podría saber si ambos números fueron encontrados.

Ejercicio 2.1.5. Vector de factoriales

Enunciado. Dado un vector Vec que contiene los primeros 15 números naturales, calcule un vector fact con sus factoriales y muéstrelo por pantalla. Considere el problema de almacenamiento a partir de factoriales grandes.

Idea clave. Usar ciclos anidados: uno para recorrer los números y otro para calcular el factorial de cada componente.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Vector de 15 números	Vec	var	entero
Vector de 15 números	fact	var	entero

Objeto	Nombre	Valor	Tipo
Contador	i	var	entero
Contador	j	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Quince	15	cte	entero

Aspecto de resolución	Orientación para el estudiante
Entrada	Identifica qué datos se leen por teclado y cuáles ya están definidos.
Proceso	Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.
Salida	Especifica qué se debe mostrar y en qué orden.
Pensamiento crítico	¿Por qué el factorial crece tan rápido? ¿Qué tipo de dato conviene usar si el resultado supera la capacidad de int?

Análisis del problema

El programa debe calcular el factorial de los primeros 15 números naturales y almacenarlos en otro vector.

Algoritmo Factoriales

Definir Vec[15] Como Entero

Definir fact[15] Como Entero

Definir i, j Como Entero

Para i <- 0 Hasta 14 Hacer

Vec[i] <- i + 1

fact[i] <- 1

Para j <- 1 Hasta Vec[i] Hacer

fact[i] <- fact[i] * j

FinPara

FinPara

Para i <- 0 Hasta 14 Hacer

Escribir Vec[i], " factorial = ", fact[i]

FinPara

FinAlgoritmo

Prueba de Escritorio_

i	Vec[i]	Multiplicación	fact[i]
0	1	1	1
1	2	1×2	2
2	3	1×2×3	6
3	4	1×2×3×4	24

Codificación en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    int Vec[15];
```

```
    long long fact[15];
```

```
    int i, j;
```

```
    for(i = 0; i < 15; i++) {
```

```
        Vec[i] = i + 1;
```

```
        fact[i] = 1;
```

```
        for(j = 1; j <= Vec[i]; j++) {
```

```

        fact[i] = fact[i] * j;
    }
}

for(i = 0; i < 15; i++) {

    printf("%d! = %lld\n", Vec[i], fact[i]);
}

return 0;
}

```

Reflexión crítica

El factorial crece muy rápido, por eso conviene usar long long en lugar de int para evitar desbordamientos.

2. Ejercicios de matrices

Ejercicio 2.2.1. Suma de componentes de una matriz

Enunciado. Halle la suma de las componentes de una matriz cuya dimensión y componentes se leen por teclado y por filas. Suponga que el número de filas N y columnas M son menores o iguales a 10.

Idea clave. Recorrer la matriz por filas con dos ciclos anidados y acumular cada componente en res.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Matriz de 10x10 componentes	mat	var	real
Contador	i	var	entero
Contador	j	var	entero
Número de filas de la matriz	N	var	entero
Número de columnas de la matriz	M	var	entero
Suma de las componentes	res	var	real
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica que datos se leen por teclado y cuales ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué se necesita un ciclo para filas y otro para columnas? ¿Qué representa cada índice?	

Análisis del problema

El programa debe leer una matriz y sumar todos sus elementos.

Algoritmo SumaMatriz

Definir mat[10][10] Como Real
Definir i, j, N, M Como Entero
Definir res Como Real

res <- 0

Leer N
Leer M

Para i <- 0 Hasta N-1 Hacer
 Para j <- 0 Hasta M-1 Hacer

 Leer mat[i][j]
 res <- res + mat[i][j]

 FinPara
FinPara

Escribir res

FinAlgoritmo

Prueba de Escritorio:

Matriz	
1	2
3	4
Operación	res
0 + 1	1
1 + 2	3
3 + 3	6
6 + 4	10
	Salida: 10

Codificación en C:

```
#include <stdio.h>
```

```
int main() {
```

```
  float mat[10][10];  
  float res = 0;  
  int i, j, N, M;
```

```
  scanf("%d", &N);  
  scanf("%d", &M);
```

```
  for(i = 0; i < N; i++) {
```

```

for(j = 0; j < M; j++) {

    scanf("%f", &mat[i][j]);
    res = res + mat[i][j];
}

printf("Suma = %.2f", res);

return 0;
}

```

Reflexión crítica

Se necesitan dos ciclos porque una matriz tiene filas y columnas.
El índice i representa las filas y j representa las columnas.

Ejercicio 2.2.2. Escritura de matriz en sentido inverso

Enunciado. Dada una matriz de N x N elementos, realice un algoritmo que recorra la matriz por filas desde la última a la primera y cada fila en sentido inverso, desde la última columna a la primera, mostrando cada elemento.

Idea clave. Inicializar los índices en la última posición y decrementar hasta llegar a la primera.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz 4 x 4	mat	var	entero
Contador	i	var	entero
Contador	j	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica que datos se leen por teclado y cuales ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Qué cambia entre recorrer una matriz normalmente y recorrerla al revés? ¿Qué riesgo aparece con los límites de los índices?	

Análisis del problema

El programa debe recorrer una matriz desde la última fila hasta la primera y desde la última columna hasta la primera, mostrando los elementos en orden inverso.

Algoritmo MatrizInversa

Definir mat[4][4] Como Entero
Definir i, j Como Entero

Para i <- 0 Hasta 3 Hacer

```

    Para j <- 0 Hasta 3 Hacer
        Leer mat[i][j]
    FinPara
FinPara

Para i <- 3 Hasta 0 Con Paso -1 Hacer

    Para j <- 3 Hasta 0 Con Paso -1 Hacer

        Escribir mat[i][j]

    FinPara

FinPara

FinAlgoritmo

```

Prueba de Escritorio:

Matriz		
1	2	
3	4	
i	j	Salida
1	1	4
1	0	3
0	1	2
0	0	1

Codificación en C:

```

#include <stdio.h>

int main() {

    int mat[4][4];
    int i, j;

    for(i = 0; i < 4; i++) {

        for(j = 0; j < 4; j++) {

            scanf("%d", &mat[i][j]);
        }
    }

    printf("Matriz invertida:\n");

    for(i = 3; i >= 0; i--) {

        for(j = 3; j >= 0; j--) {

```

```
        printf("%d ", mat[i][j]);  
    }  
  
    printf("\n");  
}  
  
return 0;  
}
```

Reflexión crítica

El recorrido cambia porque ahora los índices disminuyen en lugar de aumentar.

Debe tenerse cuidado con los límites para evitar acceder a posiciones inexistentes de la matriz.

Ejercicio 2.2.3. Máximo de una fila

Enunciado. Escriba un programa que lea una matriz de N filas y N columnas de valores enteros. Luego debe pedir el número de una fila y mostrar por pantalla el valor de la mayor componente de esa fila.

Idea clave. Inicializar max con el primer elemento de la fila seleccionada y luego comparar con el resto de componentes de esa fila.

Tabla de Objeto:

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz 4 x 4 componentes	mat	var	entero
Contador	i	var	entero
Contador	j	var	entero
Dato de entrada de fila	fil	var	entero
Máximo de la fila	max	var	entero
Uno y dos	1 y 2	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué es mejor iniciar max con un valor de la propia matriz y no con un número fijo como -1000?	

Análisis del problema

El programa debe encontrar el mayor elemento de una fila específica de la matriz.

Algoritmo MaximoFila

```
Definir mat[4][4] Como Entero
Definir fil, max Como Entero
Definir i, j Como Entero
```

```
Para i <- 0 Hasta 3 Hacer
  Para j <- 0 Hasta 3 Hacer
    Leer mat[i][j]
  FinPara
FinPara
```

```
Leer fil
```

```
max <- mat[fil][0]
```

```
Para j <- 1 Hasta 3 Hacer
```

```
  Si mat[fil][j] > max Entonces
    max <- mat[fil][j]
  FinSi
```


FinPara

Escribir max

FinAlgoritmo

Prueba de Escritorio:

j	Comparación	max
0	Inicial	4
1	7 > 4	7
2	2 > 7	7
3	9 > 7	Salida: 9

Codificación en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    int mat[4][4];  
    int i, j, fil, max;
```

```
    for(i = 0; i < 4; i++) {
```

```
        for(j = 0; j < 4; j++) {
```

```
            scanf("%d", &mat[i][j]);  
        }  
    }
```

```
    printf("Ingrese fila: ");  
    scanf("%d", &fil);
```

```
    max = mat[fil][0];
```

```
    for(j = 1; j < 4; j++) {
```

```
        if(mat[fil][j] > max) {  
            max = mat[fil][j];  
        }  
    }
```

```
    printf("Mayor = %d", max);
```

```
    return 0;  
}
```

Reflexión crítica

Es mejor iniciar max con un elemento de la matriz porque así el valor inicial siempre pertenece al conjunto de datos reales.

Ejercicio 2.2.4. Intercambiar las filas i, j de una matriz

Enunciado. Escriba un programa que intercambie las filas i y j de una matriz de enteros de N x N componentes, siendo i y j dos valores introducidos por teclado.

Idea clave. Usar una variable auxiliar para no perder información al intercambiar componente por componente.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz de 4 x 4 componentes	mat	var	entero
Dato entrada primera fila	i	var	entero
Dato entrada segunda fila	j	var	entero
Contador	k	var	entero
Variable auxiliar	aux	var	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuales ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Qué valor se perdería si se asigna directamente $mat[i][k] = mat[j][k]$ sin usar aux?	

Análisis del problema

El programa debe intercambiar dos filas de una matriz utilizando una variable auxiliar.

Algoritmo IntercambiarFilas

Definir mat[4][4] Como Entero
Definir i, j, k, aux Como Entero

Para i <- 0 Hasta 3 Hacer
 Para j <- 0 Hasta 3 Hacer
 Leer mat[i][j]
 FinPara
FinPara

Leer i
Leer j

Para k <- 0 Hasta 3 Hacer

 aux <- mat[i][k]
 mat[i][k] <- mat[j][k]
 mat[j][k] <- aux

FinPara

FinAlgoritmo

Prueba de Escritorio:

Fila 0 → 1 2 3 4

Fila 1 → 5 6 7 8

k	aux	mat[0][k]	mat[1][k]
0	1	5	1
1	2	6	2
2	3	7	3
3	4	8	4

Codificación en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    int mat[4][4];
```

```
    int i, j, k, aux;
```

```
    for(i = 0; i < 4; i++) {
```

```
        for(j = 0; j < 4; j++) {
```

```
            scanf("%d", &mat[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Ingrese fila i: ");
```

```
    scanf("%d", &i);
```

```
    printf("Ingrese fila j: ");
```

```
    scanf("%d", &j);
```

```
    for(k = 0; k < 4; k++) {
```

```
        aux = mat[i][k];
```

```
        mat[i][k] = mat[j][k];
```

```
        mat[j][k] = aux;
```

```
    }
```

```
    printf("Nueva matriz:\n");
```

```
    for(i = 0; i < 4; i++) {
```

```
        for(j = 0; j < 4; j++) {
```

```
            printf("%d ", mat[i][j]);
```

```
        }
```

```

        printf("\n");
    }

    return 0;
}

```

Reflexión crítica

La variable auxiliar evita perder datos durante el intercambio.

Si se asigna directamente un valor sobre otro, el valor original desaparece.

Ejercicio 2.2.5. Matriz traspuesta

Enunciado. Dada una matriz A de dimensión N x M, cuyas dimensiones reales y componentes se leen por teclado y por filas, calcule e imprima la matriz B de dimensión M x N, tal que $B = A^T$. Suponga que filas y columnas de A son menores o iguales a 10.

Idea clave. Al leer cada componente $A[i][j]$, almacenarla en $B[j][i]$. Luego imprimir B con dimensiones invertidas.

Tabla de Objetos:

Objeto	Nombre	Valor	Tipo
Matriz de 10x10 componentes	A	var	real
Matriz de 10x10 componentes	B	var	real
Contador	i	var	entero
Contador	j	var	entero
Numero de filas de la matriz A	N	var	entero
Numero de columnas de la matriz A	M	var	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	

Objeto	Nombre	Valor	Tipo
Entrada		Identifica que datos se leen por teclado y cuales ya estan definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica que se debe mostrar y en que orden.	
Pensamiento crítico		¿Por qué la matriz traspuesta cambia filas por columnas? ¿Cómo se modifican los límites al imprimir B?	

Análisis del problema

El programa debe generar la matriz traspuesta cambiando filas por columnas.

Algoritmo Traspuesta

Definir A[10][10] Como Real

Definir B[10][10] Como Real

Definir i, j, N, M Como Entero

Leer N

Leer M

Para i <- 0 Hasta N-1 Hacer

Para j <- 0 Hasta M-1 Hacer

Leer A[i][j]

B[j][i] <- A[i][j]

FinPara

FinPara

Para i <- 0 Hasta M-1 Hacer

Para j <- 0 Hasta N-1 Hacer

Escribir B[i][j]

FinPara

FinPara

FinAlgoritmo

Prueba de Escritorio:

Matriz A			
1	2	3	
4	5	6	
i	j	A[i][j]	B[j][i]
0	0	1	B[0][0]=1
0	1	2	B[1][0]=2
0	2	3	B[2][0]=3

1	0	4	B[0][1]=4
1	1	5	B[1][1]=5
1	2	6	B[2][1]=6
Matriz transpuesta B			
1	4		
2	5		
3	6		

Codificación en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    float A[10][10], B[10][10];
```

```
    int i, j, N, M;
```

```
    printf("Ingrese filas: ");
```

```
    scanf("%d", &N);
```

```
    printf("Ingrese columnas: ");
```

```
    scanf("%d", &M);
```

```
    for(i = 0; i < N; i++) {
```

```
        for(j = 0; j < M; j++) {
```

```
            scanf("%f", &A[i][j]);
```

```
            B[j][i] = A[i][j];
```

```
        }
```

```
    }
```

```

printf("Matriz traspuesta:\n");

for(i = 0; i < M; i++) {

    for(j = 0; j < N; j++) {

        printf("%.2f ", B[i][j]);

    }

    printf("\n");

}

return 0;
}

```

Reflexión crítica

La matriz traspuesta intercambia filas por columnas.
Por eso los índices cambian de $A[i][j]$ a $B[j][i]$.

3. Formato sugerido de entrega por ejercicio

Para cada ejercicio, el estudiante puede desarrollar la solución usando la siguiente estructura.
Este orden ayuda a que la respuesta sea clara, verificable y fácil de defender.

Numeral	Producto esperado	Descripción
1	Análisis del problema	Explicar con sus palabras qué se pide resolver y cual será la salida esperada.
2	Tabla de objetos	Completar o revisar variables, constantes, tipo de dato y rol dentro del algoritmo.
3	Diseño del algoritmo	Proponer pseudocódigo o diagrama lógico con ciclos, condiciones y asignaciones.
4	Prueba de escritorio	Validar el algoritmo con datos pequeños antes de codificar.
5	Codificación en C	Implementar en Code::Blocks cuidando índices, tipos de datos y sintaxis.
6	Reflexión crítica	Justificar decisiones: inicialización, límites, recorrido, acumuladores, banderas o variables auxiliares.

4. Errores comunes que deben evitarse

- Confundir el índice lógico del enunciado, que suele empezar en 1, con el índice de C, que empieza en 0.
- No inicializar acumuladores como res, max o variables bandera antes de usarlos.
- Usar un solo ciclo repetitivo cuando el problema requiere recorrer filas y columnas de una

matriz.

- Perder datos al intercambiar valores sin una variable auxiliar.
- No validar el tamaño máximo del vector o matriz cuando el enunciado limita N, M o n.
- Elegir un tipo de dato insuficiente para resultados grandes, como ocurre con factoriales.

5. Rúbrica de evaluación: pensamiento crítico de resolución

Puntaje total: 20 puntos. Esta rúbrica valora tanto la solución técnica como la capacidad del estudiante para justificar su razonamiento.

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Interpreta correctamente el enunciado, identifica entradas, proceso y salida, y distingue si el ejercicio usa vector o matriz.	Comprende la idea general, aunque omite algún detalle de entrada, salida o restricción.	Presenta confusión sobre lo que debe calcularse o mostrarse.	4 pts
Identificación de objetos y tipos	Reconoce variables, constantes, contadores, acumuladores, banderas o auxiliares; asigna tipos adecuados.	Identifica la mayoría de objetos, pero con pequeños errores de tipo o función.	La tabla de objetos está incompleta o no corresponde al problema.	4 pts
Diseño lógico del algoritmo	Propone ciclos, condiciones, acumulaciones o intercambios coherentes con el problema y sus límites.	La lógica principal es adecuada, aunque puede tener errores menores de límite o recorrido.	La lógica no permite resolver correctamente el ejercicio.	5 pts
Prueba de escritorio y validación	Comprueba la solución con datos pequeños y explica cómo cambian las variables paso a paso.	Realiza una prueba parcial, pero con poca explicación de los cambios internos.	No evidencia validación o la prueba no coincide con el algoritmo.	3 pts
Justificación crítica de decisiones	Argumenta por qué usa acumulador, bandera, auxiliar, inicialización o recorrido específico; detecta riesgos como índices y tipos de dato.	Justifica algunas decisiones, aunque de manera breve o incompleta.	No justifica sus decisiones o solo copia código sin explicar.	3 pts

Presentación y orden	Entrega clara, ordenada y fácil de revisar, con nombres de variables legibles y comentarios útiles.	La entrega es entendible, pero puede mejorar en organización o comentarios.	La presentación dificulta la revisión del ejercicio.	1 pt
----------------------	---	---	--	------

